

18.2.1: Bitwise Operators

[This section corresponds to K&R Sec. 2.9]

The bitwise operators operate on numbers (always integers) as if they were sequences of binary bits (which, of course, internally to the computer they are). These operators will make the most sense, therefore, if we consider integers as represented in binary, octal, or hexadecimal (bases 2, 8, or 16), not decimal (base 10). Remember, you can use octal constants in C by prefixing them with an extra 0 (zero), and you can use hexadecimal constants by prefixing them with 0x (or 0X).

The & operator performs a bitwise AND on two integers. Each bit in the result is 1 only if both corresponding bits in the two input operands are 1. For example, 0x56 & 0x32 is 0x12, because (in binary):

```

  0 1 0 1 0 1 1 0
& 0 0 1 1 0 0 1 0
-----
  0 0 0 1 0 0 1 0

```

The | (vertical bar) operator performs a bitwise OR on two integers. Each bit in the result is 1 if either of the corresponding bits in the two input operands is 1. For example, 0x56 | 0x32 is 0x76, because:

```

  0 1 0 1 0 1 1 0
| 0 0 1 1 0 0 1 0
-----
  0 1 1 1 0 1 1 0

```

The ^ (caret) operator performs a bitwise exclusive-OR on two integers. Each bit in the result is 1 if one, but not both, of the corresponding bits in the two input operands is 1. For example, 0x56 ^ 0x32 is 0x64:

```

  0 1 0 1 0 1 1 0
^ 0 0 1 1 0 0 1 0
-----
  0 1 1 0 0 1 0 0

```

The ~ (tilde) operator performs a bitwise complement on its single integer operand. (The ~ operator is therefore a unary operator, like ! and the unary -, &, and * operators.) Complementing a number means to change all the 0 bits to 1 and all the 1s to 0s. For example, assuming 16-bit integers, ~0x56 is 0xffa9:

```

~ 0 0 0 0 0 0 0 0 0 1 0 1 0 1 1 0
-----
  1 1 1 1 1 1 1 1 1 0 1 0 1 0 0 1

```

The << operator shifts its first operand left by a number of bits given by its second operand, filling in new 0 bits at the right. Similarly, the >> operator shifts its first operand right. If the first operand is unsigned, >> fills in 0 bits from the left, but if the first operand is signed, >> might fill in 1 bits if the high-order bit was already 1. (Uncertainty like this is one reason why it's usually a good idea to use all unsigned operands when working with the bitwise operators.) For example, 0x56 << 2 is 0x158:

```

  0 1 0 1 0 1 1 0 << 2
-----
  0 1 0 1 0 1 1 0 0 0

```

And `0x56 >> 1` is `0x2b`:

```

 0 1 0 1 0 1 1 0 >> 1
-----
 0 1 0 1 0 1 1

```

For both of the shift operators, bits that scroll ``off the end" are discarded; they don't wrap around. (Therefore, `0x56 >> 3` is `0x0a`.)

The bitwise operators will make more sense if we take a look at some of the ways they're typically used. We can use `&` to test if a certain bit is 1 or not. For example, `0x56 & 0x40` is `0x40`, but `0x32 & 0x40` is `0x00`:

```

 0 1 0 1 0 1 1 0      0 0 1 1 0 0 1 0
& 0 1 0 0 0 0 0 0      & 0 1 0 0 0 0 0 0
-----
 0 1 0 0 0 0 0 0      0 0 0 0 0 0 0 0

```

Since any nonzero result is considered ``true" in C, we can use an expression involving `&` directly to test some condition, for example:

```

if(x & 0x04)
    do something ;

```

(If we didn't like testing against the bitwise result, we could equivalently say `if((x & 0x04) != 0)` . The extra parentheses are important, as we'll explain below.)

Notice that the value `0x40` has exactly one 1 bit in its binary representation, which makes it useful for testing for the presence of a certain bit. Such a value is often called a *bit mask*. Often, we'll define a series of bit masks, all targeting different bits, and then treat a single integer value as a set of *flags*. A ``flag" is an on-off, yes-no condition, so we only need one bit to record it, not the 16 or 32 bits (or more) of an `int`. Storing a set of flags in a single `int` does more than just save space, it also makes it convenient to assign a set of flags all at once from one flag variable to another, using the conventional assignment operator `=`. For example, if we made these definitions:

```

#define DIRTY    0x01
#define OPEN     0x02
#define VERBOSE  0x04
#define RED      0x08
#define SEASICK  0x10

```

we would have set up 5 different bits as keeping track of those 5 different conditions (``dirty," ``open," etc.). If we had a variable

```

unsigned int flags;

```

which contained a set of these flags, we could write tests like

```

if(flags & DIRTY)
    { /* code for dirty case */ }

if(!(flags & OPEN))
    { /* code for closed case */ }

if(flags & VERBOSE)
    { /* code for verbose case */ }
else
    { /* code for quiet case */ }

```

A condition like `if(flags & DIRTY)` can be read as "if the DIRTY bit is on".

These bitmasks would also be useful for *setting* the flags. To "turn on the DIRTY bit," we'd say

```
flags = flags | DIRTY;          /* set DIRTY bit */
```

How would we "turn off" a bit? The way to do it is to leave on every bit but the one we're turning off, if they were on already. We do this with the `&` and `~` operators:

```
flags = flags & ~DIRTY;        /* clear DIRTY bit */
```

This may be easier to see if we look at it in binary. If the DIRTY, RED, and SEASICK bits were already on, flags would be 0x19, and we'd have

```

  0 0 0 1 1 0 0 1
& 1 1 1 1 1 1 1 0
-----
  0 0 0 1 1 0 0 0

```

As you can see, both the `|` operator when turning bits on and the `&` (plus `~`) operator when turning bits off have no effect if the targeted bit were already on or off, respectively.

The definition of the exclusive-OR operator means that you can use it to *toggle* a bit, that is, to turn it to 1 if it was 0 and to 0 if it was one:

```
flags = flags ^ VERBOSE;       /* toggle VERBOSE bit */
```

It's common to use the "op=" shorthand forms when doing all of these operations:

```

flags |= DIRTY;                /* set DIRTY bit */
flags &= ~OPEN;                 /* clear OPEN bit */
flags ^= VERBOSE;              /* toggle VERBOSE bit */

```

We can also use the bitwise operators to extract subsets of bits from the middle of an integer. For example, to extract the second-to-last hexadecimal "digit," we could use

```
(i & 0xf0) >> 4
```

If `i` was 0x56, we have:

```

      i          0 1 0 1 0 1 1 0
& 0x56          & 1 1 1 1 0 0 0 0
-----
              0 1 0 1 0 0 0 0

```

and shifting this result right by 4 bits gives us 0 1 0 1, or 5, as we wished. Replacing (or overwriting) a subset of bits is a bit more complicated; we must first use `&` and `~` to clear all of the destination bits, then use `<<` and `|` to "OR in" the new bits. For example, to replace that second-to-last hexadecimal digit with some new bits, we might use:

```
(i & ~0xf0) | (newbits << 4)
```

If `i` was still 0x56 and `newbits` was 6, this would give us

```

      i          0 1 0 1 0 1 1 0
& ~0xf0          & 0 0 0 0 1 1 1 1

```

```

-----
| (newbits << 4) | 0 0 0 0 0 1 1 0
                  | 0 1 1 0 0 0 0 0
                  -----
                  0 1 1 0 0 1 1 0

```

resulting in 0x66, as desired.

We've been using extra parentheses in several of these bitwise expressions because it turns out that (for the usual, hoary sort of ``historical reasons'') the precedence of the bitwise `&`, `|`, and `^` operators is low, usually lower than we'd want. (The reason that they're low is that, once upon a time, C didn't have the logical operators `&&` and `||`, and the bitwise operators `&` and `|` did double duty.) However, since the precedence of `&` and `|` (and `^`) is lower than `==`, `!=`, `<<`, and `>>`, expressions like

```
if(a & 0x04 != 0)          /* WRONG */
```

and

```
i & 0xf0 >> 4             /* WRONG */
```

would *not* work as desired; these last two would be equivalent to

```
if(a & (0x04 != 0))
i & (0xf0 >> 4)
```

and would not do the bit test or subset extraction that we wanted.

[The rest of this section is somewhat advanced and may be skipped.]

Because of the nature of base-2 arithmetic, it turns out that shifting left and shifting right are equivalent to multiplying and dividing by two. These operations are equivalent for the same reason that tacking zeroes on to the right of a number in base 10 is the same as multiplying by 10, and deleting digits from the right is the same as dividing by 10. You can convince yourself that `0x56 << 2` is the same as `0x56 * 4`, and that `0x56 >> 1` is the same as `0x56 / 2`. It's also the case that masking off all but the low-order bits is the same as taking a remainder; for example, `0x56 & 0x07` is the same as `0x56 % 8`. Some programmers therefore use `<<`, `>>`, and `&` in preference to `*`, `/`, and `%` when powers of two are involved, on the grounds that the bitwise operators are ``more efficient." Usually it isn't worth worrying about this, though, because most compilers are smart enough to perform these optimizations anyway (that is, if you write `x * 4`, the compiler might generate a left shift instruction all by itself), they're not always as readable, and they're not always correct for negative numbers.

The issue of negative numbers, by the way, explains why the right-shift operator `>>` is not precisely defined when the high-order bit of the value being shifted is 1. For signed values, if the high-order bit is a 1, the number is negative. (This is true for 1's complement, 2's complement, and sign-magnitude representations.) If you were using a right shift to implement division, you'd want a negative number to stay negative, so on some computers, under some compilers, when you shift a signed value right and the high-order bit is 1, new 1 bits are shifted in at the left instead of 0s. However, you can't depend on this, because not all computers and compilers implement right shift this way. In any case, shifting negative numbers to the right (even if the high-order 1 bit propagates) gives you an incorrect answer if there's a remainder involved: in 2's complement, 16-bit arithmetic, `-15` is `0xffff1`, so `-15 >> 1` might give you `0xffff8` shifted which is `-8`. But integer division is supposed to discard the remainder, so `-15 / 2` would have given you `-7`. (If you're having trouble seeing the way the shift worked, `0xffff1` is `1111111111110001`₂ and `0xffff8` is `111111111111000`₂. The low-order 1 bit got shifted off to the right, but because the

high-order bit was 1, a 1 got shifted in at the left.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail feedback](#)